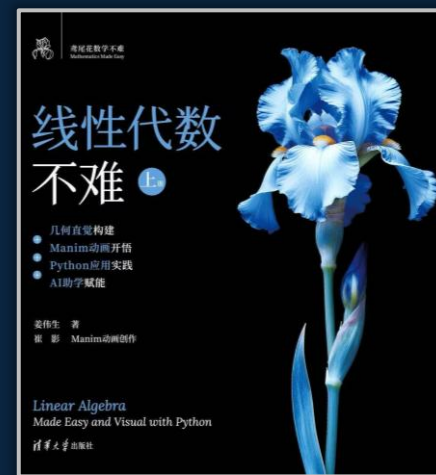


03 矩阵乘法

3.5 分块矩阵乘法

四个视角的总结，以及分块矩阵乘法的一般规律

《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>



本节你将掌握的核心技能

1 分块矩阵乘法：子矩阵形状匹配是分块相乘的前提

3 第一视角延展：A切多行，B切多列，仍然成立

5 第二视角延展：A切多列，B切多行，仍然成立

7 矩阵乘法第四视角：行向量线性组合——A切成行

2 矩阵乘法第一视角：内积——A切成行，B切成列

4 矩阵乘法第二视角：外积——A切成列，B切成行

6 矩阵乘法第三视角：列向量线性组合——B切成列

本节知识脉络

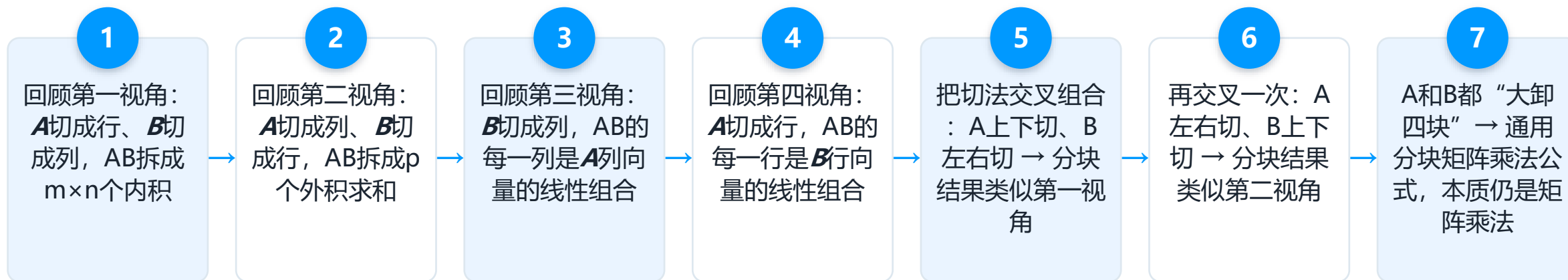


表1 · 比较矩阵乘法四个视角

$AB = C$	第一视角	第二视角	第三视角	第四视角
描述	内积	外积	列向量线性组合	行向量线性组合
分块	A 切成行 B 切成列	A 切成列 B 切成行	B 切成列	A 切成行
数学表达	$c_{ij} = \mathbf{a}^{(i)} @ \mathbf{b}_j$	$\mathbf{C}_k = \mathbf{a}_k @ \mathbf{b}^{(k)}$	$\mathbf{c}_j = \mathbf{A} \mathbf{b}_j$	$\mathbf{c}^{(i)} = \mathbf{a}^{(i)} \mathbf{B}$

表1 · 比较矩阵乘法四个视角

$AB = C$	第一视角	第二视角	第三视角	第四视角
意义	A 的第 <i>k</i> 行和 <i>B</i> 的第 <i>k</i> 列的内积	A 的第 <i>k</i> 列和 <i>B</i> 的第 <i>k</i> 行的外积矩阵求和	A 的列向量根据 <i>B</i> 的每一列加权组合	<i>B</i> 的行向量根据 <i>A</i> 的每一行加权组合
形状	由 <i>A</i> 的行数× <i>B</i> 的列数决定	每个外积结果是 <i>m</i> × <i>n</i> ，与最终矩阵相同	A 的列向量与 <i>C</i> 的列向量维数相同	<i>B</i> 的行向量与 <i>C</i> 的行向量维数相同

A切成行，B切成列

$$\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{i,j}$$

- 把左侧矩阵**A**切成行，右侧矩阵**B**切成列，就把矩阵乘法**AB**写成了 $m \times n$ 个矩阵乘法
- 每个矩阵乘法 $\mathbf{a}^{(i)} @ \mathbf{b}_j = c_{i,j}$ ，得到的是结果**C**中的一个元素——这就是矩阵乘法的第一视角，内积

图1 • 矩阵乘积 AB ， A 切成行， B 切成列

- A 有 m 个行向量 $\mathbf{a}^{(1)} \dots \mathbf{a}^{(m)}$ ， B 有 n 个列向量 $\mathbf{b}_1 \dots \mathbf{b}_n$ ，中间维度 p 被“消去”
- 结果 C 的每一个元素 $c_{i,j}$ ，都是 A 的第 i 行与 B 的第 j 列做内积的结果

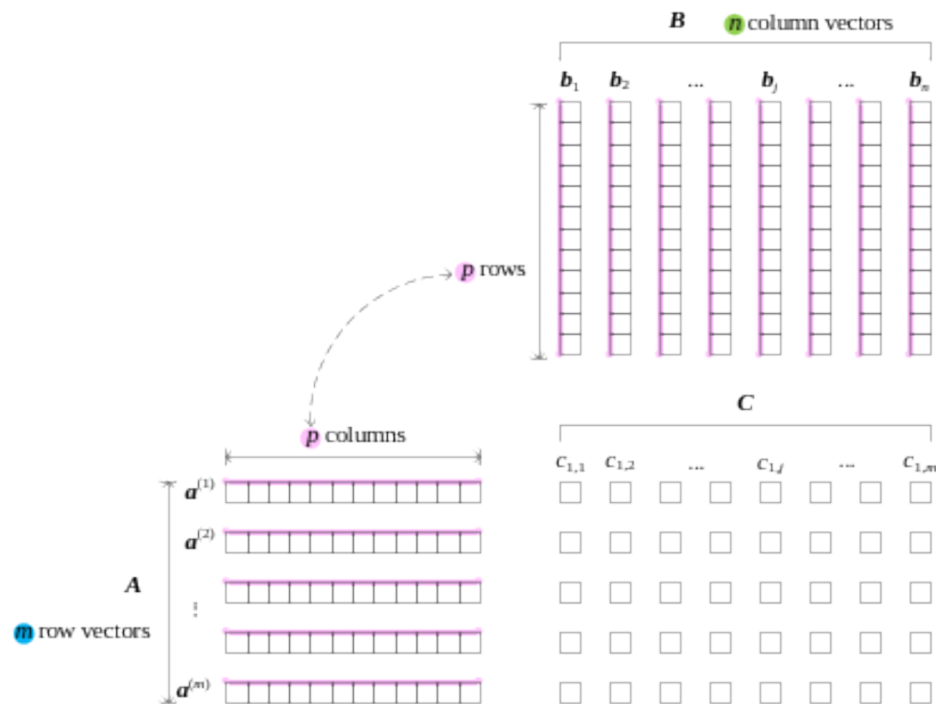


图 1. 矩阵乘积 AB ， A 切成行， B 切成列

A切成列，B切成行

$$\mathbf{a}_k @ \mathbf{b}^{(k)} = \mathbf{C}_k$$

- 把左侧矩阵**A**切成列，右侧矩阵**B**切成行，就把矩阵乘法**AB**写成了p个矩阵乘法的叠加
- 每个矩阵乘法 $\mathbf{a}_k @ \mathbf{b}^{(k)} = \mathbf{C}_k$ ，得到的是和结果**C**形状完全一致的矩阵；把p个 $\mathbf{C}_k$ 叠加便得到**C**——这就是矩阵乘法的第二视角，外积

图2 • 矩阵乘积 AB ， A 切成列， B 切成行

- p 对应 A 的列数，也是 B 的行数，就是矩阵乘法 AB 中被“消去”的维度
- 每个外积 $\mathbf{a}_k @ \mathbf{b}^{(k)}$ 都是 $m \times n$ 矩阵，与最终结果 C 形状相同； p 个外积矩阵逐元素相加得到 C

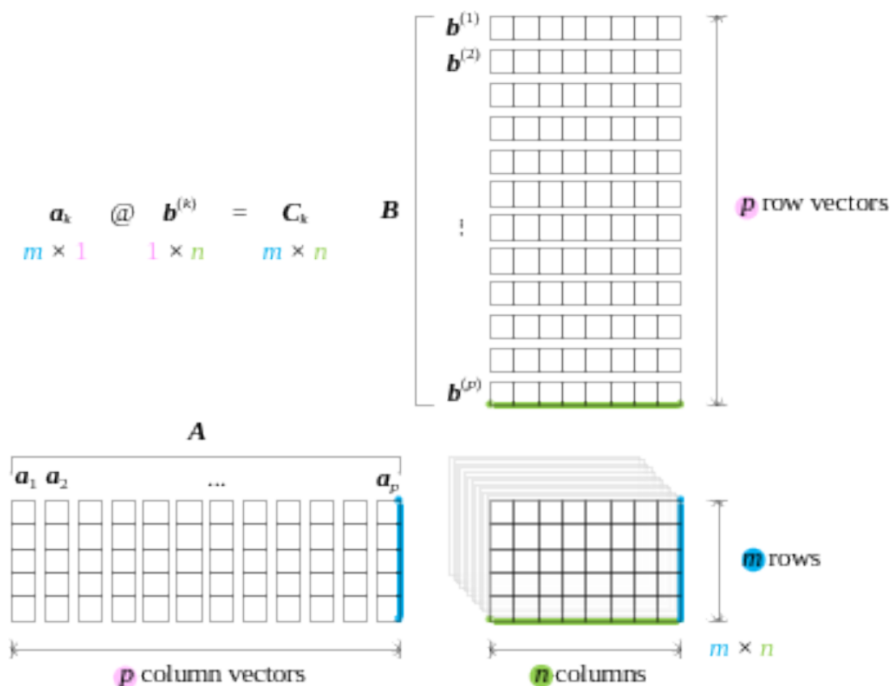


图 2. 矩阵乘积 AB ， A 切成列， B 切成行

B切成列向量

$$A[b_1 \ b_2 \ \cdots b_p] = [c_1 \ c_2 \ \cdots c_p]$$

- 把右侧矩阵**B**切成列，**C**的每一列都是**A**列向量的线性组合：若 $Ab_1 = c_1$, $Ab_2 = c_2$ $Ab_p = c_p$ p个等式，可以合成上式
- 这个视角格外重要，本书之后的投影运算中经常见到这种展开方法

图3 • 矩阵乘积 AB ， B 切成列

- A 是 $m \times p$ 矩阵， B 的每一列 b_j 是 $p \times 1$ 列向量； Ab_j 得到的 c_j 是 $m \times 1$ 列向量
- 反过来看， C 的每一列 c_j 都可以由 A 的 p 个列向量按 b_j 的分量加权组合得到

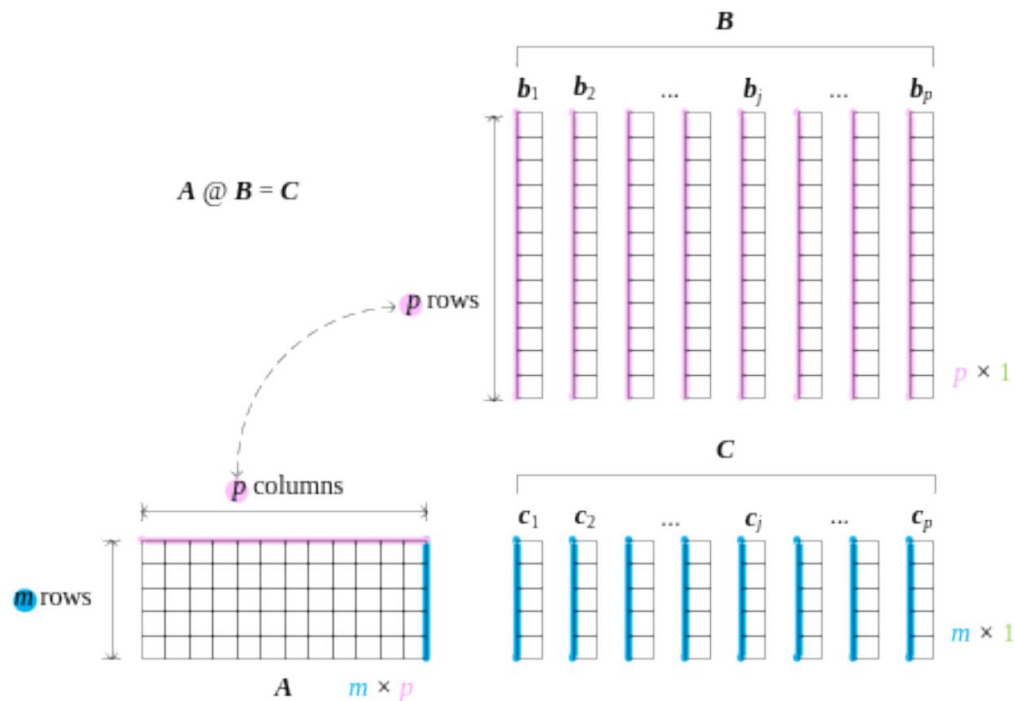


图 3. 矩阵乘积 AB ， B 切成列

B切一刀，结果不变

类似地，B先左右切一刀

$$AB = A[B_1 \ B_2] = [AB_1 \ AB_2]$$

- 把**B**左右切成两块**B**₁、**B**₂，**AB**就展开成[**AB**₁ **AB**₂]——每一块单独计算，再左右拼接
- 这其实就是列向量线性组合视角的“粗粒度”版本：把一组列向量打包成一块，规律完全一样

图4 • 矩阵乘积 AB ，将 B 左右切一刀

- B_1 、 B_2 分别是 B 左右两部分的列子块，各自与 A 相乘得到 AB_1 、 AB_2
- 把 AB_1 、 AB_2 左右拼接，正好得到完整的 AB

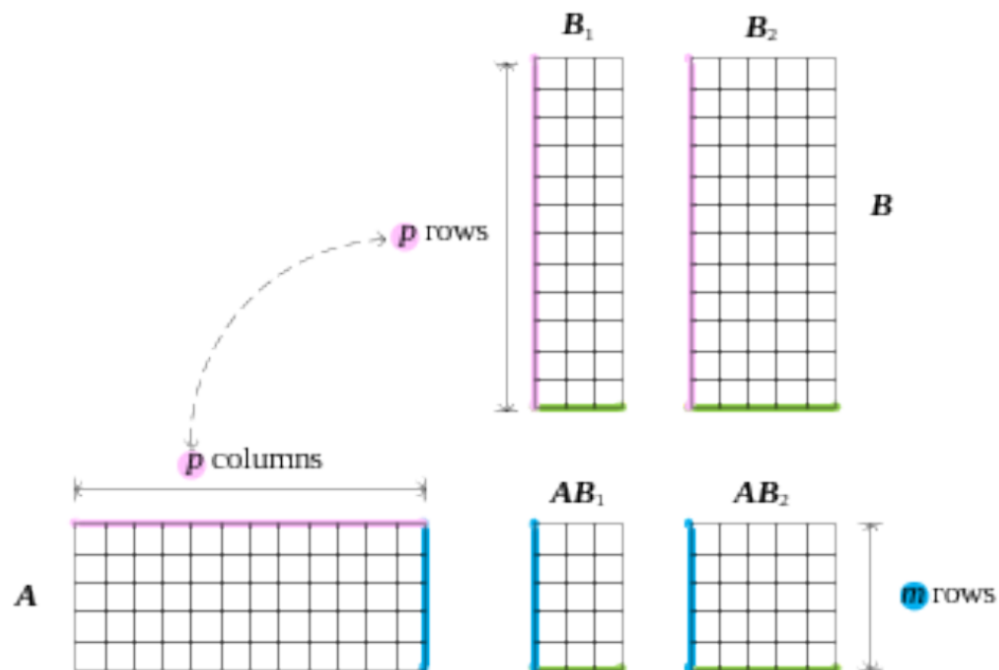


图 4. 矩阵乘积 AB ，将 B 左右切一刀

图5 • 矩阵乘积 AB ，将 B 左右切若干刀

- 把 B 左右切成 B_1 、 B_2 、 B_3任意多块，规律不变： $AB = [AB_1 \ AB_2 \ AB_3 \ \dots]$
- 无论切几刀，只要每一块的行数与 A 的列数匹配，分块乘法的结果与整体计算完全一致

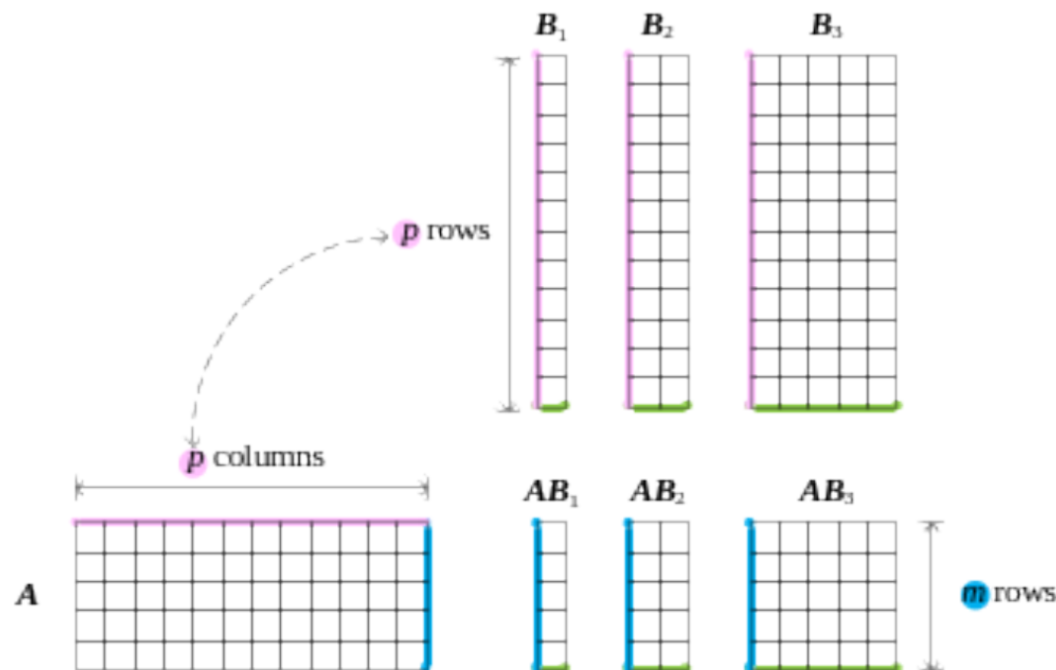


图 5. 矩阵乘积 AB ，将 B 左右切若干刀

A切成一组行向量

$$[\mathbf{a}^{(1)}; \mathbf{a}^{(2)}; \vdots; \mathbf{a}^{(m)}] \mathbf{B} = [\mathbf{a}^{(1)} \mathbf{B}; \mathbf{a}^{(2)} \mathbf{B}; \vdots; \mathbf{a}^{(m)} \mathbf{B}]$$

- 把左侧矩阵 $\mathbf{A}$ 切成行， $\mathbf{C}$ 的每一行都是 $\mathbf{B}$ 行向量的线性组合
- 也可以从“合成”角度反过来看：分别算出 $\mathbf{a}^{(1)} \mathbf{B}$ 、 $\mathbf{a}^{(2)} \mathbf{B}$再按顺序上下排列，就拼出完整的 $\mathbf{C}$

图6 • 矩阵乘积 AB ， A 切成行

- A 有 m 个行向量 $\mathbf{a}^{(1)} \dots \mathbf{a}^{(m)}$ ，每一个行向量单独与 B 相乘，结果正是 C 对应的那一行
- 这与图1的 $\mathbf{a}^{(i)} @ B = \mathbf{c}^{(i)}$ 完全一致，只是这里强调的是“逐行计算 C ”这个整体过程

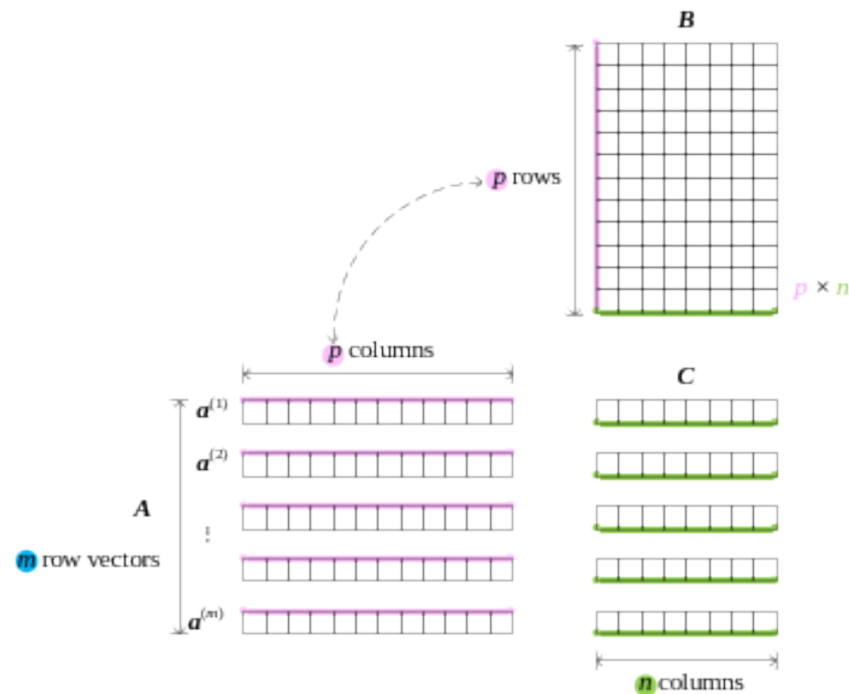


图 6. 矩阵乘积 AB ， A 切成行

A切一刀，结果不变

将A先上下切一刀

$$AB = [A_1; A_2]B = [A_1B; A_2B]$$

- 把A上下切成两块A₁、A₂，AB就展开成[A₁B, A₂B]——每一块单独计算，再上下拼接
- 这同样是行向量线性组合视角的“粗粒度”版本：把一组行向量打包成一块，规律完全一样

图7 • 矩阵乘积 AB ，将A上下切一刀

- A_1 、 A_2 分别是 A 上下两部分的行子块，各自与 B 相乘得到 A_1B 、 A_2B
- 把 A_1B 、 A_2B 上下拼接，正好得到完整的 AB

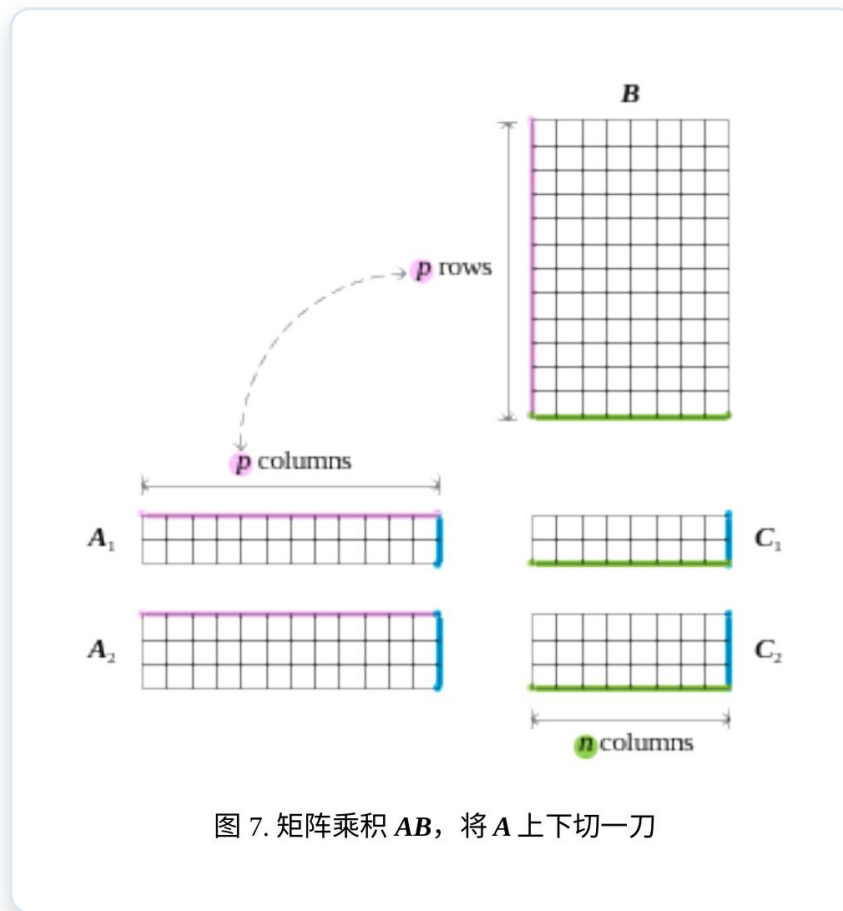


图 7. 矩阵乘积 AB ，将 A 上下切一刀

图8 • 矩阵乘积 AB ，将A上下切几刀

- 把 A 上下切成 A_1 、 A_2 、 A_3任意多块，规律不变： $AB = [A_1B, A_2B, A_3B;]$
- 无论切几刀，只要每一块的列数与 B 的行数匹配，分块乘法的结果与整体计算完全一致

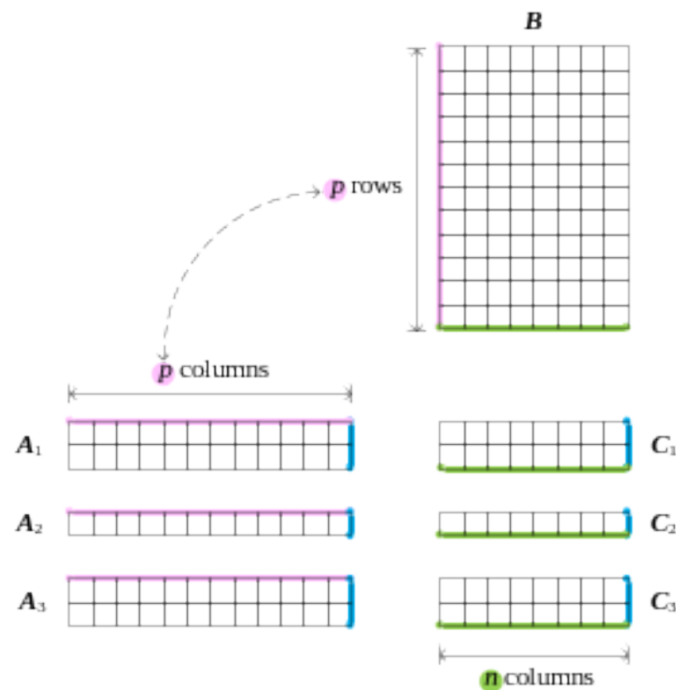


图 8. 矩阵乘积 AB ，将 A 上下切几刀

A上下切, B左右切

$$AB = [A_1; A_2][B_1 \ B_2] = [A_1B_1 \ A_1B_2; A_2B_1 \ A_2B_2]$$

- 上下分块的 A 乘左右分块的 B , 结果展开为一个 2×2 的分块矩阵, 每一块都是 A 的子块与 B 的子块相乘
- 可以把 A_1 、 A_2 看成 A 的两个“元素”, B_1 、 B_2 看成 B 的两个“元素”——这个视角类似矩阵乘法的第一视角

图9 • 矩阵乘积 AB ， A 上下分块， B 左右分块

- A_1 的列数、 A_2 的列数都等于 B_1 的行数、 B_2 的行数——子矩阵形状必须匹配，才能相乘
- 结果被分成 $C_{1,1}$ 、 $C_{1,2}$ 、 $C_{2,1}$ 、 $C_{2,2}$ 四块，每一块都是一对子矩阵的乘积

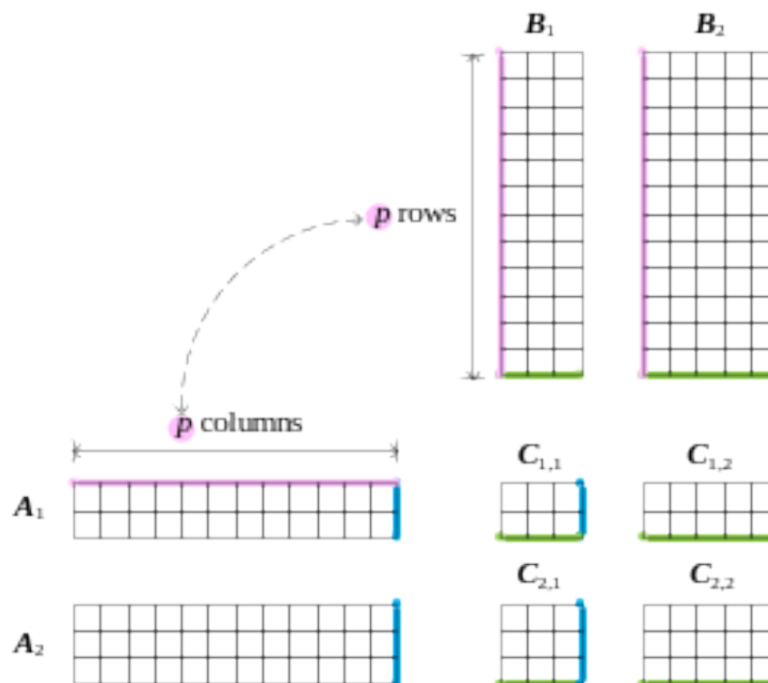


图 9. 矩阵乘积 AB ， A 上下分块， B 左右分块

A左右切，B上下切

$$AB = [A_1 \ A_2][B_1; B_2] = A_1B_1 + A_2B_2$$

- 左右分块的 A 乘上下分块的 B ，结果展开为两个子矩阵乘积之和—— A_1 列数等于 B_1 行数， A_2 列数等于 B_2 行数
- 这类似前面讲到的矩阵乘法第二视角：把整体乘法拆成若干块乘积，再逐块累加求和

图10 • 矩阵乘积 AB ， A 左右分块， B 上下分块

- A 被左右切成 A_1 、 A_2 两个列子块， B 被上下切成 B_1 、 B_2 两个行子块，两两配对相乘
- A_1B_1 与 A_2B_2 形状都与最终结果 C 相同，两者逐元素相加即得 C ——与第二视角的外积求和如出一辙

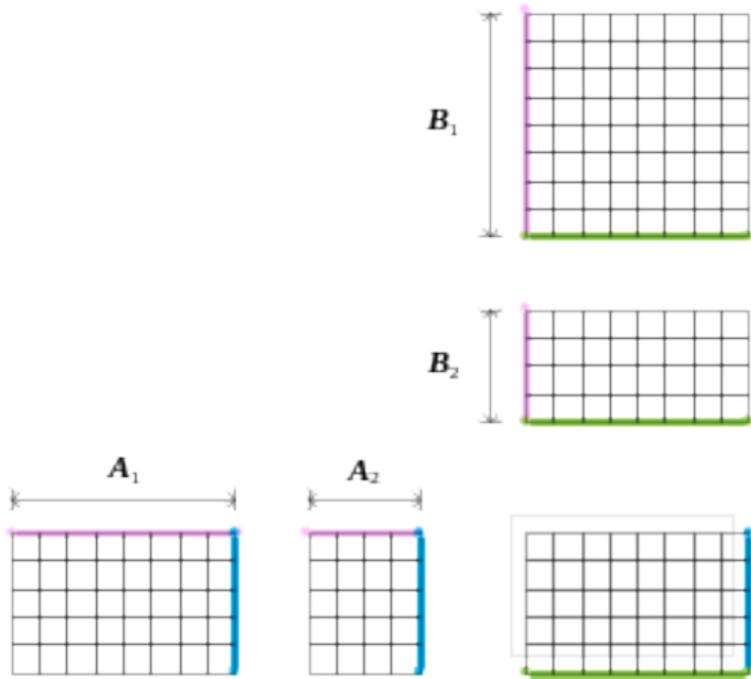


图 10. 矩阵乘积 AB ， A 上下分块， B 左右分块

切得更细也成立

图11 • 矩阵乘积 AB ， A 左右切几块， B 上下切几块

- 把 A 和 B 切得更细碎一些，规律依旧成立： $AB = A_1B_1 + A_2B_2 + A_3B_3 + \dots$
- 无论切几块，只要 A 的每一列子块与 B 对应的行子块形状匹配，逐块相乘再求和就能得到完整结果

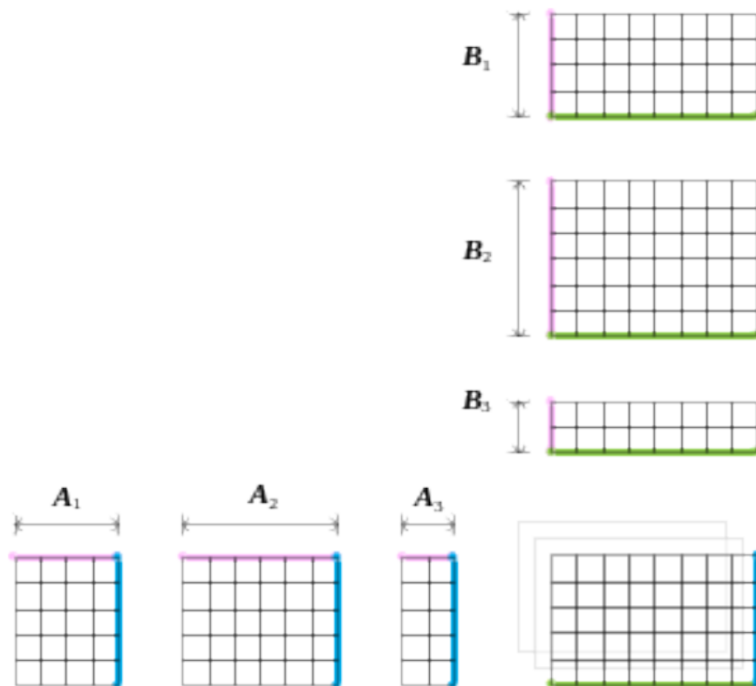


图 11. 矩阵乘积 AB ， A 上下切几块， B 左右切几块

A和B都“大卸四块”

$$AB = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix} \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix} = \begin{bmatrix} A_{1,1}B_{1,1} + A_{1,2}B_{2,1} & A_{1,1}B_{1,2} + A_{1,2}B_{2,2} \\ A_{2,1}B_{1,1} + A_{2,2}B_{2,1} & A_{2,1}B_{1,2} + A_{2,2}B_{2,2} \end{bmatrix}$$

- **A**和**B**都上下左右分块， $A_{1,1}$ 、 $A_{1,2}$ 、 $A_{2,1}$ 、 $A_{2,2}$ 的列数分别等于 $B_{1,1}$ 、 $B_{2,1}$ 、 $B_{1,2}$ 、 $B_{2,2}$ 的行数
- 这相当于两个 2×2 矩阵相乘，只是每个“元素”换成了子矩阵，结果**C**也是 2×2 分块——这也相当于矩阵乘法的第一视角

图12 • A 和 B 都上下左右分块

- A 被分成 2×2 共4块, B 也被分成 2×2 共4块; C 的每一块都是 A 对应行子块与 B 对应列子块乘积之和
- 本书有关矩阵乘法的专题到此结束; 但矩阵乘法的应用才刚刚开始——之后的投影、最小二乘、特征值等内容都建立在这四个视角之上

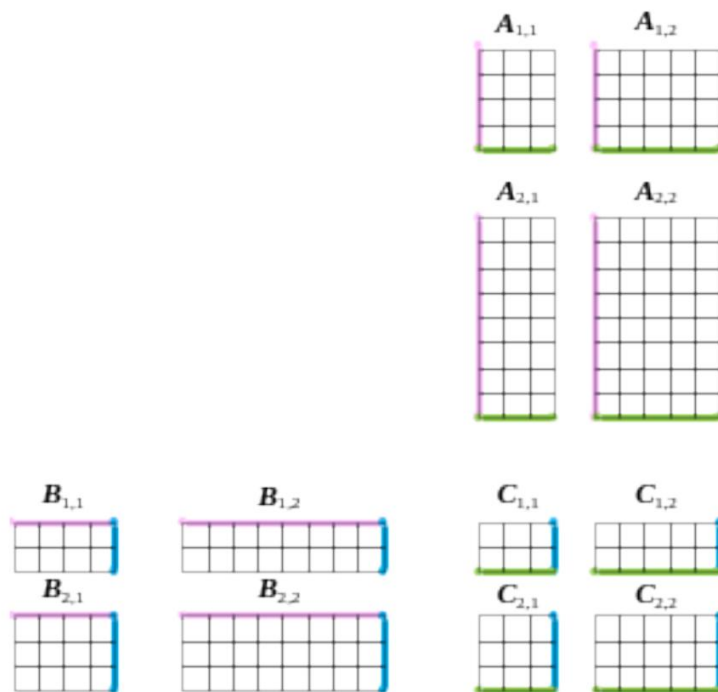


图 12. A 和 B 都上下左右分块

分块矩阵乘法的工具与应用

切片取子块 NumPy中 $A[:m1, :]$ 、 $A[m1:, :]$ 对应上下分块， $A[:, :p1]$ 、 $A[:, p1:]$ 对应左右分块，切片本身几乎不产生额外开销

np.block() `np.block([[A11, A12], [A21, A22]])`可以把四个子矩阵直接拼接成一个完整矩阵，是分块思路在代码中的直接体现

einsum统一表达 `numpy.einsum`可以用一个字符串表达式统一描述内积、外积、列向量线性组合、行向量线性组合等所有视角，本节课后Q1即安排了这一自学任务

并行与分布式计算 大规模矩阵乘法常被切成若干块，分配到不同CPU核心或GPU、甚至不同计算节点分别计算，再汇总——分块矩阵乘法是并行线性代数算法的理论基础

深度学习中的batch matmul 神经网络中常见的批量矩阵乘法（batch matmul）本质上就是把一批小矩阵沿新维度“打包”分块，逐块独立相乘

四个视角，一个矩阵乘法

内积、外积、列向量线性组合、行向量线性组合，本质上都是分块矩阵乘法在不同切法下的具体形态

- 无论怎么切 A 和 B ，只要子矩阵的形状相互匹配，分块计算的结果永远和整体计算完全一致
- 把矩阵乘法拆成子块来理解和计算，是贯穿本书后续投影、最小二乘、特征分解等内容的重要工具

本节关键术语

分块矩阵 *Block Matrix* — 把一个矩阵按行、按列或同时按行列切成若干子矩阵

第一视角·内积 A 切成行, B 切成列, $c_{ij} = a(i) @ b_j$

第三视角·列向量线性组合 B 切成列, $c_j = Ab_j$, C 的每列是 A 列向量的加权和

分块外积式组合 A 左右分块、 B 上下分块, $AB = A1B1 + A2B2$, 类似第二视角

np.block() *NumPy*函数, 把若干子矩阵按分块结构拼接为完整矩阵

子矩阵形状匹配 分块相乘时, 左侧子块的列数必须等于右侧对应子块的行数

第二视角·外积 A 切成列, B 切成行, $C_k = a_k @ b(k)$, p 个外积矩阵求和

第四视角·行向量线性组合 A 切成行, $c(i) = a(i)B$, C 的每行是 B 行向量的加权和

分块内积式组合 A 上下分块、 B 左右分块, 结果是 2×2 分块矩阵, 类似第一视角

einsum *numpy.einsum*, 用爱因斯坦求和约定统一表达各种矩阵/张量运算

配套代码与延伸阅读

延伸阅读 • `numpy.einsum`官方文档

<https://numpy.org/doc/2.2/reference/generated/numpy.einsum.html>

延伸阅读 • `numpy.block`官方文档

了解如何用子矩阵拼接出完整的分块矩阵

回顾 • LA_03_01 ~ LA_03_04系列notebook

矩阵乘法四个视角对应的完整代码实现，建议结合本节表1一并复习

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q9

Q1 请自学爱因斯坦求和约定 (Einstein Summation Convention)，参考

<https://numpy.org/doc/2.2/reference/generated/numpy.einsum.html>,

Q3 把矩阵 $B = \begin{bmatrix} 1, 4, 7, 10 \\ 2, 5, 8, 11 \\ 3, 6, 9, 12 \end{bmatrix}$ 左右切成两块 B_1 、 B_2 (各2列)，用Python分别计算 AB_1 、 AB_2 并拼接，与 $A @ B$ 比较。

Q5 写一个函数`block_matmul_2x2(A1,A2,B1,B2)`，实现 $AB = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \begin{bmatrix} B_1 & B_2 \end{bmatrix}$ 的分块计算，返回 2×2 分块结果。

Q7 用`np.block()`把手工算出的 $C_{1,1}$ 、 $C_{1,2}$ 、 $C_{2,1}$ 、 $C_{2,2}$ 四个子块拼接成完整矩阵 C ，并验证与 $A @ B$ 是否相等。

Q9 写一个函数`check_block_shapes(A1,A2,B1,B2)`，检查参与分块乘法的四个子矩阵形状是否两两匹配，不匹配则抛出`ValueError`。

Q2 给定 $A = [1, 2, 3; 4, 5, 6]$, $B = [1, 4; 2, 5; 3, 6]$ ，分别用第一、二、三、四视角自定义函数计算 $A @ B$ ，验证四种结果是否完全一致。

Q4 把矩阵 $A = [1, 2, 3; 4, 5, 6; 7, 8, 9; 10, 11, 12]$ 上下切成两块 A_1 、 A_2 (各2行)，用Python分别计算 A_1B 、 A_2B 并拼接，与 $A @ B$ 比较。

Q6 写一个函数`block_matmul_sum(A1,A2,B1,B2)`，实现 $AB = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \begin{bmatrix} B_1 & B_2 \end{bmatrix} = A_1B_1 + A_2B_2$ ，并与`np.dot(A,B)`比较结果。

Q8 构造一对 4×4 矩阵 A 、 B ，把它们都切成 2×2 共4块，用图12的公式手工验证任意一块 (如 $C_{1,2}$) 是否正确。

习题 Q10 – Q18

Q10 用`numpy.einsum('ij,jk->ik', A, B)`复现矩阵乘法 $A @ B$ ，并与内积、外积两种自定义函数的结果比较。

Q12 写一个函数`random_block_partition(A, row_cuts, col_cuts)`，按给定的切分点把矩阵A切成若干子块，返回子块列表。

Q14 探索：如果把A、B都切成 3×3 共9块（而不是 2×2 共4块），写出对应的通用分块乘法公式，并用Python验证。

Q16 构造两个分块对角矩阵（block-diagonal，即 $A_{1,2}=A_{2,1}=0$ ），验证它们相乘时非对角块是否恒为零，并解释原因。

Q18 探索：给定形状为(8,8)的矩阵A、B，模拟把它们各自切成4块分配给4个“计算节点”，分别计算后再汇总得到C，估算这种并行方案能节省多少单节点计算量。

Q11 用`einsum`实现矩阵乘法的第二视角（外积求和）：尝试写出等价的`einsum`表达式，并解释其含义。

Q13 用`time`模块比较分块计算AB（先切块再拼接）与直接调用 $A @ B$ 在大规模矩阵（如 400×400 ）上的运行时间。

Q15 写一个函数`visualize_block_partition(A, row_cuts, col_cuts)`，用`matplotlib`画出矩阵A按给定切分线分块后的示意图。

Q17 写一个函数`compare_all_views(A, B)`，用第一、二、三、四视角以及 2×2 分块视角共5种方式计算 $A @ B$ ，统计各自运行时间并绘制柱状图比较。

习题 Q19 – Q26

Q19 写一个函数`block_matmul_general(A_blocks, B_blocks)`，输入任意形状的分块矩阵（以二维列表存储子块），返回完整的分块乘法结果。

Q21 探索批量矩阵乘法：构造形状为 $(\text{batch}, 2, 3)$ 与 $(\text{batch}, 3, 2)$ 的两组矩阵，用NumPy的`@`运算符或`np.matmul()`一次性计算所有batch的乘积，并与逐个循环计算的结果比较。

Q23 结合表1，写一个函数`summarize_views(A, B)`，打印出四种视角下矩阵乘法的中间结果（内积值列表、外积矩阵列表、列组合结果、行组合结果），帮助复习本节内容。

Q25 用Python分别实现“先分块再拼接”和“直接整体相乘”两种方式计算大矩阵乘积，比较二者结果的数值误差是否在浮点精度范围内。

Q20 用Python验证：把单位矩阵 I 切成 2×2 分块（ $I_{1,1}$ 、 $I_{2,2}$ 为小单位矩阵， $I_{1,2}$ 、 $I_{2,1}$ 为零矩阵），验证 $I @ B$ 分块计算后是否等于 B 本身。

Q22 写一个函数`block_transpose(A_blocks)`，验证分块矩阵转置的规律： $(AB)^T$ 的分块是否等于把每个子块转置后再整体转置分块结构（子块位置对调、子块自身转置）。

Q24 探索：如果 A 是 2×2 分块的下三角分块矩阵（ $A_{1,2}$ 为零矩阵）， B 是 2×2 分块的下三角分块矩阵，验证 AB 是否仍然是下三角分块矩阵，并从图12公式推导原因。

Q26 结合本节所学，写一份简短总结（可以是注释形式），说明矩阵乘法四个视角与分块矩阵乘法之间的关系，并给出一个你认为最直观的记忆方法。

10 个可深入探讨的 Prompt (1/2)

- 1 请解释矩阵乘法的四个视角（内积、外积、列向量线性组合、行向量线性组合）本质上为什么都是分块矩阵乘法的特殊形式。
- 2 请解释为什么分块矩阵乘法要求子矩阵形状两两匹配，如果形状不匹配会出现什么问题？
- 3 请解释 $AB = \begin{bmatrix} A1 \\ A2 \end{bmatrix} \begin{bmatrix} B1 & B2 \end{bmatrix}$ （A上下分块、B左右分块）这种切法为什么类似矩阵乘法的第一视角。
- 4 请解释 $AB = \begin{bmatrix} A1 & A2 \end{bmatrix} \begin{bmatrix} B1 \\ B2 \end{bmatrix} = A1B1 + A2B2$ （A左右分块、B上下分块）这种切法为什么类似矩阵乘法的第二视角。
- 5 请解释A和B都“大卸四块”时的通用分块乘法公式，并说明它与 2×2 矩阵相乘的规则有何相似之处。

10 个可深入探讨的 Prompt (2/2)

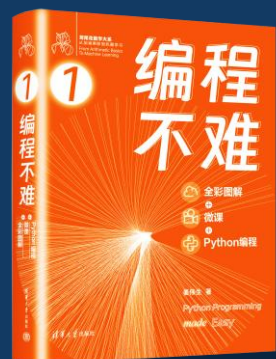
- 6 请解释`np.block()`函数的作用，以及它与手工用切片拼接子矩阵相比有什么优势。
- 7 请解释`numpy.einsum`如何用一个统一的字符串表达式描述矩阵乘法的不同视角，并举一两个具体例子。
- 8 请解释分块矩阵乘法在并行计算或分布式计算中的应用价值，为什么大规模矩阵运算常常要先分块？
- 9 请解释深度学习中的`batch matmul`（批量矩阵乘法）与本节分块矩阵乘法之间的联系。
- 10 请结合本章矩阵乘法的四个视角与分块矩阵乘法，谈谈理解“同一个运算可以有多种等价视角”对学习线性代数的意义。

数学

数学基础

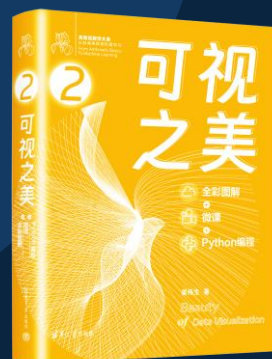
线性代数

概率统计

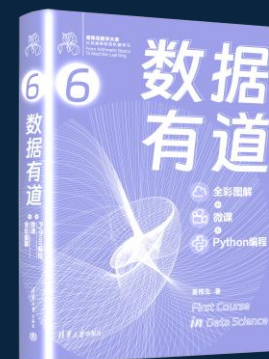
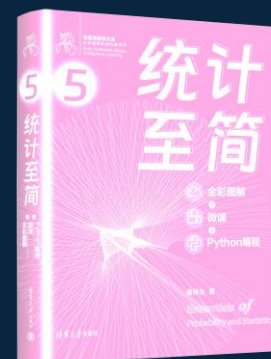
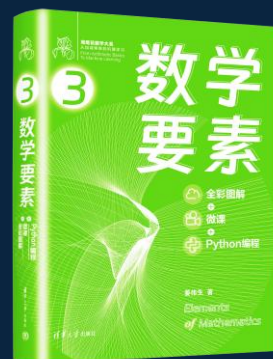


Python编程

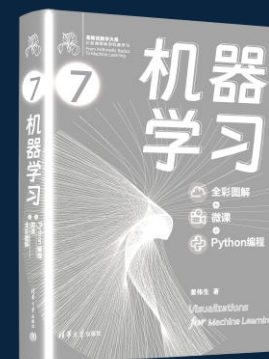
工具



数据可视化



数据分析
图和网络



回归、降维
分类、聚类

实践

